

MODULE 12 · PART 3 - CTES AND RECURSION

Cycles, Depth and Cost

What happens when the data is not a tree, and nothing tells you.

PART 3 of 8 MODULE 12 of 26 RUNS about 34 min

Why this gets a module

- ◆ **It is the only failure in this course that can take down a platform**, rather than producing a wrong number.
- ◆ **And the only one with no error message**, at any point, on any engine that lacks a default cap.
- ◆ **The defences are all cheap**, and none of them is default behaviour on the engines most warehouses run.
- ◆ **So it is an hour of material about four lines of SQL**, and the four lines are the ones nobody writes.

THE TAKEAWAY

Every other module has been about getting an answer right. This one is about the query finishing at all, which is a lower bar and a harder one to guarantee.

What an unguarded upward walk does

```
1 -- ancestors of 4471, unguarded
2 -- pass 1    4471
3 -- pass 2    4498
4 -- pass 3    4471
5 -- pass 4    4498
6 -- ...
7 -- pass 4000000      4 million rows, still going
8
9 -- the same table, walked DOWNWARD from the roots,
10 -- finishes in under a second and omits 42 nodes.
```

10 One table, two directions, two completely different failures. Only one of them is loud.

THE TAKEAWAY

No single pass is slow. The query is not struggling, it is succeeding, repeatedly, which is why monitoring on slow steps does not catch it.

The shapes a cycle takes

SHAPE	WHAT IT LOOKS LIKE, AND HOW IT BEHAVES
Self-loop	A node is its own parent. One row. Grows linearly and forever.
Two-cycle	A and B name each other. Kestrel's case. Linear, and slow to notice.
Long cycle	A to B to C to A. Harder to spot by eye, and behaves identically.
Cycle with a subtree	The loop also has children. Grows geometrically, and fails fast.
Diamond, not a cycle	Two paths reach one node. Legitimate, and often mistaken for a loop.

THE TAKEAWAY

The last row matters as much as the others. A node reachable by two routes is a graph, not a broken tree, and pruning it loses real data.

Fast failures and slow ones

- ◆ **A cycle with branching fails within minutes.** Geometric growth fills memory quickly and somebody notices.
- ◆ **A bare cycle can run all night.** One row per pass is cheap, and the result grows steadily rather than explosively.
- ◆ **So the loud failure is the lucky one.** It gets investigated the same day and fixed the same week.
- ◆ **The quiet one produces a result.** Capped at twenty, it returns in seconds and looks exactly like a correct walk.

THE TAKEAWAY

Kestrel's cycle is the quiet kind. It has produced plausible, capped, wrong output on every walk that met it for two and a half years.

One bad row turns a terminating query into a non-terminating one. **With no error, and no warning.**

Where cycles come from

- ◆ **A correction applied in the wrong direction.** Somebody reparents A under B, having already reparented B under A.
- ◆ **A bad load.** Two source systems merged, and an identifier meant something different in each of them.
- ◆ **A genuine many-to-many** modelled as a tree. A product in two categories, or an employee with two managers.
- ◆ **None of these is exotic**, and none of them announces itself. The table looks completely normal.

THE TAKEAWAY

Kestrel's category cycle came from a merchandising correction in March 2024. It has been there ever since, and every query that met it was capped rather than fixed.

The cycle, and which way it bites

```
1 category_key | parent_key | category_name
2 -----+-----+-----
3          4471 |          4498 | Chilled Desserts
4          4498 |          4471 | Ambient Desserts
5
6 -- walking DOWN from the roots: never reaches either.
7 --   the pair, and the 40 categories beneath them,
8 --   are silently missing from the result.
9 -- walking UP from any of those 42: loops forever.
```

- 6 A node in a cycle has its parent inside the cycle, so a downward walk can never arrive at it.
- 9 Upward, the same two rows are a loop with no exit. Same data, opposite failure.

THE TAKEAWAY

Nothing in either row is invalid. Both keys exist, neither is null, and the defect is a property of the pair rather than of either one.

The second cycle, in the org chart

```
1 employee_key | manager_key | loaded_from
2 -----+-----+-----
3      88104 |      88251 | TUPE transfer 2025
4      88251 |      88377 | TUPE transfer 2025
5      88377 |      88104 | TUPE transfer 2025
```

5 A three-node loop, from a transfer load where manager identifiers meant something different in the source system.

THE TAKEAWAY

Longer than the category cycle and no different in behaviour. Nobody spots a three-cycle by reading rows, which is the argument for a probe rather than a review.

Two cycles, two lessons

- ◆ **The category one came from an edit.** A human correction applied without checking what the reverse edge already said.
- ◆ **The employee one came from a load.** Two systems merged, and an identifier meant one thing in each.
- ◆ **Neither source system is wrong.** Both are internally consistent, and the cycle exists only in the merged result.
- ◆ **Which is why the check belongs in the warehouse.** It is the first place the whole graph exists at once.

THE TAKEAWAY

A cycle created by merging two correct sources cannot be found in either source. The warehouse is where it becomes visible, so the warehouse is where it gets checked.

Sometimes the data is **meant** to be a graph. Then a cycle is not a defect.

Tree, or graph

- ◆ **A tree has one parent per node and no loops.** A cycle in a tree is a defect and somebody should be told.
- ◆ **A graph has neither constraint.** A cycle is ordinary, and the walk still has to be stopped.
- ◆ **The same SQL handles both.** Which is why teams stop distinguishing them, and then alert on nothing.
- ◆ **Decide which you have, and write it down,** because the guard looks identical and the alerting does not.

THE TAKEAWAY

The path check is required in both cases. The nightly cycle probe is only meaningful for the one that is supposed to be a tree.

Walking a graph on purpose

```
1 WITH RECURSIVE reach AS (  
2   SELECT product_key AS start_key, product_key AS node,  
3         ARRAY[product_key] AS visited, 0 AS hops  
4   FROM dim_product WHERE product_key = 55210  
5 UNION ALL  
6   SELECT r.start_key, s.substitute_key,  
7         r.visited || s.substitute_key, r.hops + 1  
8   FROM reach r  
9   JOIN bridge_substitution s ON s.product_key = r.node  
10  WHERE NOT s.substitute_key = ANY(r.visited)  
11         AND r.hops < 3)  
12 SELECT DISTINCT node, MIN(hops) FROM reach GROUP BY node;
```

- 10 The same path check, and here it is not detecting a defect. It is stopping a legitimate loop.
- 11 Plus a hop limit, because on a graph the answer is 'within three steps' rather than 'to the bottom'.

THE TAKEAWAY

A substitution network is cyclic by design. The guard is identical and the interpretation is opposite: nothing here needs fixing.

The recursive term cannot ask **have I been here before**. Module eleven explained why.

The path array, doing real work

```
1 WITH RECURSIVE walk AS (  
2   SELECT category_key, parent_key, 1 AS depth,  
3         ARRAY[category_key] AS path  
4   FROM dim_category WHERE parent_key IS NULL  
5   UNION ALL  
6   SELECT c.category_key, c.parent_key, w.depth + 1,  
7         w.path || c.category_key  
8   FROM dim_category c  
9   JOIN walk w ON c.parent_key = w.category_key  
10  WHERE NOT c.category_key = ANY(w.path))  
11 SELECT * FROM walk;
```

10 The whole defence, in one line. Do not visit a node that is already on this row's own route.

THE TAKEAWAY

The path is per row, not global. Two different branches may both legitimately contain the same node, and each row only refuses to revisit its own ancestors.

Why the path check is the right shape

- ◆ **A cycle is a node appearing twice on one route.** That is the definition, and the path is exactly that route.
- ◆ **It terminates the affected branch and nothing else.** The rest of the tree is walked normally and completely.
- ◆ **A global visited-set would be wrong,** because a node reachable by two legitimate paths would be dropped from one of them.
- ◆ **And it costs an array comparison per row,** which is real, and far cheaper than the alternative.

THE TAKEAWAY

The third point is why UNION is not a substitute. Deduplicating globally removes genuine second visits along with the cyclic ones.

Flagging rather than pruning

```
1  SELECT c.category_key, c.parent_key, w.depth + 1,  
2         w.path || c.category_key,  
3         c.category_key = ANY(w.path) AS is_cycle  
4  FROM dim_category c  
5  JOIN walk w ON c.parent_key = w.category_key  
6  WHERE NOT w.is_cycle
```

- 3 Compute the flag on the row that closes the loop, so it appears in the output.
- 6 Then stop at the flagged row rather than before it. One extra row per cycle, and the evidence is in the result.

THE TAKEAWAY

Pruning hides the cycle. Flagging shows you which node closed the loop, which is what somebody will need in order to fix the data.

The CYCLE clause

```
1 WITH RECURSIVE walk AS (  
2     SELECT category_key, parent_key FROM dim_category  
3     WHERE parent_key IS NULL  
4     UNION ALL  
5     SELECT c.category_key, c.parent_key  
6     FROM dim_category c  
7     JOIN walk w ON c.parent_key = w.category_key  
8 ) CYCLE category_key SET is_cycle USING route  
9 SELECT * FROM walk WHERE NOT is_cycle;
```

8 One clause. It builds the route array, sets the flag, and stops the walk when it sees a repeat.

THE TAKEAWAY

Exactly the previous two slides, generated. PostgreSQL 14 and later. Redshift, Snowflake, SQL Server and MySQL have nothing equivalent.

Cycle handling, by engine

ENGINE	WHAT IT GIVES YOU
PostgreSQL 14+	CYCLE clause, plus SEARCH. Both build the array for you.
Oracle	CONNECT BY NOCYCLE, and a CONNECT_BY_ISCYCLE pseudocolumn.
SQL Server	Nothing. MAXRECURSION 100, which truncates rather than detects.
Redshift, Snowflake, BigQuery	Nothing, and no default cap. Build the path by hand.
MySQL	A depth limit only. Truncates, and errors when it does.

THE TAKEAWAY

Two engines detect cycles, two truncate, and two do neither. The manual path check is the only technique that works on all six.

Reading the guards out loud

WRITTEN

SAID

`NOT c.key = ANY(w.path)`

do not go somewhere this row has already been

`w.depth < 20`

and give up after twenty steps regardless

`c.key = ANY(w.path) AS is_cycle`

but mark the row that closes a loop

`WHERE NOT w.is_cycle`

and stop after the marked row, not before it

`CYCLE key SET is_cycle USING route`

all four of the above, in one clause

THE TAKEAWAY

Four lines of SQL and four sentences of English. If you cannot say which of the four a query has, it probably has none of them.

An analyst adds cycle protection to a category walk, carrying a path built by concatenating `category_name` and checking that the next node's name is not already in it. The query now terminates.

Merchandising report that 340 categories are missing from the output. All of them sit beneath a parent whose subtree contains a category with the same name as one of its ancestors, such as 'Seasonal' appearing at two levels.

What is wrong?

- ☐ A The CYCLE clause should be used instead, since a hand-built check cannot be relied upon.
- ☐ B The path should be a string rather than an array, so that partial matches are impossible.
- ☐ C The depth cap is set too low, and the missing categories sit below it.
- ☐ D The path is built from names, which are not unique, so legitimate branches are pruned as if they were cycles.

◆ PAUSE AND DECIDE

What is wrong?

A Use the CYCLE clause

Not available on Kestrel's warehouse, and it would work by checking the key. The problem is the column chosen, not the technique.

C The cap is too low

Would drop whole levels uniformly. These are missing by name collision, which is not a property of depth.

B Use a string path

Backwards. A concatenated string is what causes this, and it adds substring matches on top. The array is the safer structure.

D Names are not unique

A repeated name is not a repeated node. The check must compare keys, which are unique, rather than labels, which are not.

THE SIGNAL

"Seasonal" appearing at two levels." A hierarchy repeats labels all the time. Only the key identifies a node, and a cycle check that compares anything else prunes real branches.

A depth cap does not detect anything. It **bounds the damage**, which is a different job.

The containment check itself

- ◆ **Postgres writes it as $x = \text{ANY}(\text{array})$** , and Redshift and Snowflake have array containment functions under other names.
- ◆ **It is a linear scan of the array**, so the cost grows with depth. On a six-level tree that is six comparisons.
- ◆ **Keep the array narrow.** Carry keys, not names, not whole rows. An integer array is cheap and a text array is not.
- ◆ **Where arrays are awkward, a delimited string works**, with delimiters on both ends so that 44 does not match 447l.

THE TAKEAWAY

The delimiter detail is the one that bites. Searching for '447l' inside '|44|7l|' finds nothing, and searching for '44' inside '|447l|' finds a match that is not there.

The string version, done carefully

```
1 -- wrong: substring collisions
2 WHERE w.path NOT LIKE '%' || c.category_key || '%'
3
4 -- right: delimited on both ends
5     w.path || c.category_key || '|' AS path,
6 WHERE w.path NOT LIKE '%| ' || c.category_key || '| %'
```

- 2 Key 44 matches inside 4471. Legitimate branches are pruned, and the symptom is scenario one's.
- 6 Every key wrapped in delimiters, so a match is a whole key and never part of one.

THE TAKEAWAY

An array avoids this entirely, which is why it is the default recommendation. Use the string form only where arrays are unavailable, and delimit both ends.

What the path check costs

- ◆ **One array append and one containment scan per row.** Both are proportional to the current depth.
- ◆ **So the total is roughly rows times average depth,** which on a shallow wide tree is negligible and on a deep one is not.
- ◆ **And the arrays are carried in the working table,** so a deep walk moves more data between passes than a shallow one.
- ◆ **On a six-level tree it is free.** On a fifty-level one, measure it before assuming.

THE TAKEAWAY

For every hierarchy in this course the cost is irrelevant. It is worth knowing the shape of it so that the one time it matters, you recognise it.

A cheaper check, for deep walks

```
1 SELECT c.category_key, c.parent_key, w.depth + 1,  
2        w.path || c.category_key,  
3        w.seen | (1 << (c.category_key % 63)) AS seen  
4 FROM dim_category c  
5 JOIN walk w ON c.parent_key = w.category_key  
6 WHERE (w.seen >> (c.category_key % 63)) & 1 = 0  
7        OR NOT c.category_key = ANY(w.path)
```

6 A bitmask as a cheap first filter. Constant-time, and it can produce false positives, so the array check still runs.

THE TAKEAWAY

A two-stage check: a constant-time filter that is usually right, then the exact one only when the cheap filter fires. Worth it on deep walks and nowhere else.

When to reach for that

- ◆ **Almost never.** On a six-level tree the array check is already free and the bitmask is pure complexity.
- ◆ **When the walk is hundreds of levels deep**, which happens with chains rather than hierarchies. A supply chain, a citation graph.
- ◆ **Or when the fan-out is enormous**, so that the array is being copied millions of times per pass.
- ◆ **And measure first.** The cost is almost always the join rather than the containment check.

THE TAKEAWAY

Included because somebody will meet a deep walk and assume nothing can be done. Something can, and it is a last resort rather than a default.

Cap, or path check

DEPTH CAP

Bounds the damage

- One comparison per row
- Stops everything, cycle or not
- Silently truncates a deep branch
- Works on every engine

PATH CHECK

Detects the cause

- An array comparison per row
- Stops only the affected branch
- Leaves the rest of the tree intact
- Needs the path carried

THE TIEBREAKER

The cap is cheap and blunt, and it silently truncates legitimate depth. The path check is precise and costs more. Write both, because each covers the other's failure.

Choosing the cap

- ◆ **Well above the real depth.** Kestrel's tree is six levels, so twenty is sensible and a hundred is still fine.
- ◆ **A tight cap truncates and lies.** Set it to six and the day somebody adds a seventh level, a branch vanishes with no error.
- ◆ **Assert on the cap being reached,** so it is visible when it fires. A cap that fires silently is worse than no cap.
- ◆ **SQL Server's default of 100 is exactly this trade,** made for you, and it does at least raise an error when it fires.

THE TAKEAWAY

A cap is a fuse. A fuse that blows without anybody noticing has not protected you, it has just moved the failure somewhere quieter.

Making the cap visible

```
1 WITH RECURSIVE walk AS (  
2   ...  
3   WHERE w.depth < 20  
4   AND NOT c.category_key = ANY(w.path))  
5 SELECT (SELECT COUNT(*) FROM walk WHERE depth >= 20) AS hit_cap,  
6        (SELECT COUNT(*) FROM walk WHERE is_cycle) AS cycles_found,  
7        (SELECT COUNT(*) FROM walk) AS rows_walked,  
8        (SELECT COUNT(*) FROM dim_category) AS nodes;
```

- 5 Any row at the cap means the walk was truncated. On a six-deep tree that should be impossible.
- 6 And any row flagged as a cycle means the data is not a tree. Both should be zero, every night.

THE TAKEAWAY

Four numbers. Three of them should never change and one of them should always be zero. That is what turns a guard into a check.

A cap means every cycle you have ever had was **silently absorbed**.

Finding every cycle, deliberately

```
1 WITH RECURSIVE probe AS (  
2   SELECT category_key AS start_key, category_key AS node,  
3         ARRAY[category_key] AS path, false AS looped  
4   FROM dim_category  
5   UNION ALL  
6   SELECT p.start_key, c.category_key,  
7         p.path || c.category_key,  
8         c.category_key = ANY(p.path)  
9   FROM probe p  
10  JOIN dim_category c ON c.parent_key = p.node  
11  WHERE NOT p.looped AND array_length(p.path, 1) < 50  
12 SELECT DISTINCT start_key, path FROM probe WHERE looped;
```

- 2 Every node is a starting point, not just the roots. A cycle that no root reaches is still a cycle.
- 12 The output is the loops themselves, with the route each one takes. That is what a fix needs.

THE TAKEAWAY

Expensive, and you run it nightly rather than per query. It answers a different question: not 'did this walk hit a cycle' but 'does this table contain one'.

Detection is a data quality job

- ◆ **Run it on a schedule, against the dimension**, not inside the forty queries that read it.
- ◆ **Alert on a non-zero result**. The expected answer is no rows, every night, for years.
- ◆ **Do not remove the guards once you have it**. A check runs nightly and a cycle can arrive at nine in the morning.
- ◆ **Keep the loop's route in the alert**, because the fix needs to know which edge to reverse, not merely that a loop exists.

THE TAKEAWAY

Guards protect the query. Checks protect the data. A team with only guards has queries that always finish and a warehouse whose hierarchy is quietly wrong.

Kestrel's category cycle is discovered in September 2026 by an engineer reading this course's material. It has been in `dim_category` since March 2024. No query has ever failed because of it.

Every recursive walk in the warehouse carries a depth cap of 20, which is why. The team asks whether the caps should be removed so that future cycles surface immediately.

What should they do?

- A Remove the caps. A query that hangs is a defect that gets investigated, which is better than one that hides.
- B Keep the caps, and add a scheduled check that looks for cycles explicitly and alerts on any it finds.
- C Add a foreign key from `parent_key` to `category_key`, which prevents a node from referencing a missing parent.
- D Raise the caps to 100, so that a genuine deep hierarchy is never truncated and cycles become obvious.

◆ PAUSE AND DECIDE

What should they do?

A Remove the caps

Trades a silent defect for an outage, and the outage arrives at the worst possible time. The guard is not the problem.

C Add a foreign key

Worth having, and it prevents dangling parents rather than loops. Both rows in this cycle reference keys that exist.

B Keep caps, add a check

The cap bounds the damage and the check finds the cause. They do different jobs and neither substitutes for the other.

D Raise the caps to 100

Makes a runaway take five times as long before it stops. It does not make a cycle obvious, because the query still terminates quietly.

THE SIGNAL

"No query has ever failed because of it." That is the cap working, and it is why the defect is two and a half years old. A guard that hides is doing half its job.

A runaway recursive query does not just fail. It **takes the cluster with it.**

A truncated walk returns rows, quickly, and looks **exactly like a correct one.**

Detecting truncation

```
1 SELECT COUNT(*) FILTER (WHERE depth >= 20) AS hit_the_cap,  
2     COUNT(*) FILTER (WHERE is_cycle) AS closed_a_loop,  
3     COUNT(DISTINCT category_key) AS nodes_reached,  
4     (SELECT COUNT(*) FROM dim_category) AS nodes_in_table,  
5     MAX(depth) AS deepest  
6 FROM walk;
```

- 1 Non-zero means a branch was cut. On a six-level tree with a cap of twenty, this can only mean a loop.
- 4 Nodes reached should equal nodes in the table, unless some nodes are genuinely unreachable from any root.

THE TAKEAWAY

Five numbers, one query, and it runs against a walk you were doing anyway. Two of them should be zero and two should be equal, every single night.

Why silent truncation is the worse failure

- ◆ **A runaway gets investigated.** Somebody is paged, an incident is written, and the cause is found within a day.
- ◆ **A truncation gets published.** The report renders, the numbers look reasonable, and nobody has a reason to look.
- ◆ **And the error is not uniform.** One branch is wrong and the rest are right, so totals reconcile approximately.
- ◆ **Which is the shape this whole course keeps returning to.** The loud failure is the cheap one.

THE TAKEAWAY

Module five's silent frame, module seven's filtered denominator, and this. The same pattern: correct at the boundaries, wrong in one place, and nothing says so.

What actually happens while it runs

- ◆ **The result grows every pass**, and nothing reclaims it, because the query has not finished and might yet need it.
- ◆ **Memory fills, then it spills**, and spilling is when a slow query becomes an expensive one for everybody else.
- ◆ **A branching cycle grows geometrically**. Two children per node and twenty passes is a million rows from one bad edge.
- ◆ **A non-branching cycle grows linearly**, which is worse in one way. It can run for hours before anybody notices.

THE TAKEAWAY

The geometric case fails fast and loudly. The linear case runs all night, produces a plausible-sized result, and is the one that reaches a report.

What will stop it, by engine

ENGINE	THE BACKSTOP
PostgreSQL	statement_timeout, and nothing else. It is not set by default.
Amazon Redshift	Query monitoring rules, and a statement timeout. Both are configuration.
Snowflake	STATEMENT_TIMEOUT_IN_SECONDS, which defaults to two days.
SQL Server	MAXRECURSION 100 by default, raising an error when it fires.
MySQL	cte_max_recursion_depth, defaulting to 1,000.

THE TAKEAWAY

Only two of these stop a runaway by default, and one of the two defaults to two days. Everywhere else, the backstop is something somebody had to configure.

Set a statement timeout

- ◆ **It is the only engine-level defence that does not depend on the query author.** Every other guard is something somebody remembered.
- ◆ **Set it per role or per queue**, so an analyst's ad hoc session is bounded more tightly than a scheduled load.
- ◆ **It does not fix anything.** The query still fails, and it fails at the timeout rather than at the cause.
- ◆ **But it converts an outage into an error**, which is the entire point. One query fails instead of the cluster degrading.

THE TAKEAWAY

Guards are per query, checks are per table, and the timeout is per platform. The third is the only one that protects you from code you have not reviewed.

A new analyst writes an unguarded recursive walk over `dim_category` on the Redshift warehouse at 18:40. It runs overnight, spills heavily, and by 06:00 the cluster has no free disk. Two scheduled loads fail and three dashboards time out.

The query is killed at 07:15. The post-incident review asks what would have prevented it. No statement timeout is configured on the warehouse.

What is the most important change?

- ☐ A Kill the query sooner, by adding an alert on long-running statements so somebody is paged within the hour.
- ☐ B Change the walk to use `UNION` instead of `UNION ALL`, which would have deduplicated the repeated rows.
- ☐ C Set a statement timeout at the warehouse, then add the depth cap and path check to the query itself.
- ☐ D Move ad hoc analysis to a separate cluster, so that a bad query cannot affect scheduled workloads.

◆ PAUSE AND DECIDE

What is the most important change?

A Alert and kill sooner

Depends on somebody being awake at two in the morning. It shortens the outage rather than preventing it.

B Use UNION

Module eleven. It deduplicates whole rows, and the walk carries no depth column here, so it might have worked and by accident.

C Timeout first, then guards

The timeout is the only defence that does not depend on the author. The guards then fix this query specifically.

D Separate the cluster

Genuinely good architecture and a quarter of work. It also lets the same query consume the ad hoc cluster instead.

THE SIGNAL

"No statement timeout is configured." A twelve-hour outage from one query, on a platform with no time limit, is a configuration finding before it is a SQL one.

Runaway signatures

WHAT YOU OBSERVE	WHAT IT PROBABLY IS
Query running for hours, low CPU, growing disk	A bare cycle. One cheap row per pass, forever.
Query fails in minutes with memory pressure	A cycle with branching. Geometric growth.
Query finishes fast, output is short	Truncated at the cap. Check whether the cap fired.
Query finishes, output far larger than the tree	A diamond or a cycle, counted repeatedly.
Query returns only the anchor rows	The recursive join is the wrong way round. Module eleven.

THE TAKEAWAY

Five observations, five different causes, and only two of them involve the query failing. The other three complete and give you something.

Looking at one while it runs

```
1 -- what is it doing
2 SELECT pid, now() - query_start AS running_for, state, query
3   FROM pg_stat_activity
4  WHERE query ILIKE '%RECURSIVE%'
5  ORDER BY query_start;
6
7 -- and stop it
8 SELECT pg_cancel_backend(pid);
```

- 2 Running for, rather than CPU. A bare cycle uses very little CPU and will not appear in a top-consumers list.
- 8 Cancel before terminate. Cancel ends the query and leaves the session, which is usually what you want.

THE TAKEAWAY

The equivalent exists on every platform under a different name. What matters is knowing to sort by duration rather than by resource use.

After you have killed it

- ◆ **Do not rerun it.** The obvious next step, and it will do exactly the same thing on exactly the same data.
- ◆ **Run the cycle probe instead**, which is bounded, and find out whether the data is the cause before touching the query.
- ◆ **Then add the guards**, and check whether the same shape exists in other walks against the same table.
- ◆ **And write the finding down.** The next person to meet it will be somebody who was not in the incident.

THE TAKEAWAY

The most common response to a killed recursive query is to rerun it out of hours, which converts a visible incident into an invisible one.

Every guard in this module is a
workaround. The cycle is **a defect in the
data.**

Why no constraint prevents this

- ◆ **A foreign key checks that the parent exists.** Both rows in Kestrel's cycle reference keys that exist, so it passes.
- ◆ **A CHECK constraint sees one row.** It can forbid a node being its own parent, and nothing longer.
- ◆ **Acyclicity is a property of the whole table,** and no standard constraint can express a property of the whole table.
- ◆ **So it has to be a trigger, or a check on load, or a scheduled assertion.** Those are the three options, everywhere.

THE TAKEAWAY

This is not a gap somebody forgot to fill. Checking acyclicity on every write means walking the tree on every write, which is why nobody offers it as a constraint.

The three places to catch it

WHERE	WHAT IT COSTS, AND WHAT IT CATCHES
A trigger on write	Walks the tree on every update. Catches everything, and makes writes slow.
A validation on load	Runs once per batch. Catches everything the batch introduced, and nothing between batches.
A scheduled assertion	Runs nightly. Catches everything eventually, and lets a cycle live for up to a day.
In the query, as a guard	Runs constantly. Catches nothing. Bounds the damage only.

THE TAKEAWAY

Most teams end up with the last row and believe they have the third. The guard is in the code and the check is not, and the difference is invisible until somebody looks.

Following the discovery, an engineer proposes preventing the problem at source by adding a database constraint to `dim_category` so that a cycle can never be inserted again.

The table is loaded nightly from the merchandising system by a full replace. Kestrel's warehouse is Redshift, and the operational replica is PostgreSQL 16.

What should the review conclude?

- A No constraint can express acyclicity. Validate on load, and keep a scheduled assertion as a backstop.
- B Add a CHECK constraint that `parent_key` is not equal to `category_key`, which forbids the self-reference case.
- C Add a foreign key from `parent_key` to `category_key`, which makes the parent-child relationship enforced.
- D Make `parent_key` NOT NULL with a sentinel root value, so that every node has a defined place in the tree.

◆ PAUSE AND DECIDE

What should the review conclude?

- A

Validate on load
Acyclicity is a whole-table property. A full nightly replace is the natural place to check it, once, before the table is published.
- B

CHECK parent <> self
Catches loops of length one and nothing else. Kestrel's cycle is length two, so this constraint passes on it.
- C

A foreign key
Worth having and orthogonal. It prevents a parent that does not exist, and a cycle is made entirely of parents that do.
- D

NOT NULL with a sentinel
Removes the roots, which the anchor depends on, and makes every cycle harder to see rather than preventing any.

THE SIGNAL

"Loaded nightly by a full replace." That is the moment the whole table exists in one place and can be checked as a whole, which is exactly what acyclicity requires.

The four layers

01

Platform

A statement timeout.
Protects you from
queries nobody has
reviewed.

02

Query

A depth cap and a
path check. Bounds
damage, and stops the
affected branch.

03

Data

A scheduled assertion.
Finds the cycle and
names the edge that
closes it.

04

Source

Validation on load, or a
fix upstream. The only
layer that removes the
problem.

THE TAKEAWAY

Four layers, and only the last one fixes anything. The first three exist because the fourth is often somebody else's system and somebody else's quarter.

The nightly assertion, packaged

```
1 CREATE OR REPLACE VIEW dq_category_cycles AS
2 WITH RECURSIVE probe AS (
3     SELECT category_key AS start_key, category_key AS node,
4           ARRAY[category_key] AS path, false AS looped
5     FROM dim_category
6   UNION ALL
7     SELECT p.start_key, c.category_key, p.path || c.category_key,
8           c.category_key = ANY(p.path)
9     FROM probe p JOIN dim_category c ON c.parent_key = p.node
10    WHERE NOT p.looped AND array_length(p.path,1) < 50)
11 SELECT DISTINCT start_key, path FROM probe WHERE looped;
```

- 1 A view, so the check has a name, an owner and a definition anybody can read. Not a query in somebody's history.

THE TAKEAWAY

Naming it is most of the value. A check with a name gets scheduled, gets alerted on, and survives the person who wrote it leaving.

What the alert should say

- ◆ **The route.** Not that a cycle exists, but which keys form it and in what order.
- ◆ **The affected subtree size,** so somebody can judge how much reporting is wrong while it is unfixed.
- ◆ **When it first appeared,** if you keep history, because that identifies the load or the edit that introduced it.
- ◆ **Not just a count.** 'One cycle detected' is an alert somebody will acknowledge and not act on.

THE TAKEAWAY

An alert that names two keys gets fixed. An alert that says a check failed gets muted, and then the check is worse than nothing.

A query that finishes **proves nothing**. It may simply have been stopped.

Part 3, in four sentences

- ◆ **A CTE is a named scope, and materialisation is a decision about execution** that must never decide correctness.
- ◆ **Recursion is iteration**, with a working table holding one generation, which is why state has to ride on the rows.
- ◆ **Cycles are a property of pairs**, which is why no per-row constraint can prevent them.
- ◆ **And every guard here bounds damage rather than fixing data**. The fix is always upstream.

THE TAKEAWAY

Three modules, four sentences. The rest was worked examples of what happens when each of them is forgotten.

The confusions, settled

BELIEF

UNION protects against loops

A depth cap detects cycles

A foreign key prevents cycles

The CYCLE clause is standard, so it is available

A query that finishes met no cycle

A path check is a global visited set

THE TRUTH

Only if the rows are identical. A depth column defeats it.

It bounds them. It detects nothing and says nothing.

It prevents missing parents. A cycle has none.

PostgreSQL 14+ and little else. Not Redshift or Snowflake.

It may have been capped. Finishing proves nothing.

It is per row. Two branches may legitimately share a node.

THE TAKEAWAY

Rows two and five are the same belief from two directions, and together they are why Kestrel's cycle survived two and a half years.

Five habits

- ◆ **Compare keys, never labels.** A repeated name is not a repeated node, and scenario one is what happens when it is.
- ◆ **Flag the cycle rather than pruning it,** so the evidence reaches the output and somebody can act on it.
- ◆ **Assert that the cap never fires,** because a guard that fires silently has moved the failure rather than prevented it.
- ◆ **Run a cycle probe on a schedule,** against the dimension, and alert on any result at all.
- ◆ **Ask for a statement timeout once,** at the platform. It is the only guard that covers code you will never see.

THE TAKEAWAY

The first two are about this query. The last three are about every query, including the ones somebody writes next Tuesday without reading any of this.

Part 4 takes the techniques from Part 2 and asks them **time-shaped questions**.